

Advanced Python for Data Science

Week(2): Concurrency & Asynchronous Workflows

Tuesday, 23rd June 2026, 09:30-12:45

Dr. Ghadeer Mobasher

A Tale of Two Kitchens

Imagine you order three dishes at a restaurant.

Kitchen A : One Chef, One Order at a Time

Cook dish 1 (10 min) → THEN dish 2 (10 min) → THEN dish 3 (10 min) =
30 minutes

Kitchen B : Chef Starts All 3, Works While Each Waits

Start all 3 dishes cooking → check on each while others simmer = ~12
minutes

This is exactly what happens inside your programs.

Every time your code downloads a file, queries a database, or calls an API, it 'waits' ; like a dish simmering on the stove.

A program that handles this waiting badly wastes enormous time.

By the end of today, you'll know how to write programs that behave like Kitchen B.

Real-World Hook: Your Weather App

A weather app on your phone fetches data for 5 cities you follow.

Without concurrency:



With concurrency (`asyncio`):



Same 5 network calls.

5× faster , not because the network got faster, but because your program stopped wasting time waiting for one response before starting the next.

Why Concurrency Matters ?

Modern software constantly waits on the outside world :

Network Requests

API calls, web scraping, microservices

Database Queries

Reads/writes that hit disk or remote servers

File I/O

Reading/writing logs, datasets, uploads

User Input / Sensors

Waiting on events that may take seconds

The CPU is blazing fast, but the **network, disk, and database** are slow and unpredictable.
Concurrency lets your CPU start the next task instead of sitting idle .

Three Tools, Three Jobs

Python gives us three different approaches , each solves a different problem.

Multi -threading

Many workers sharing ONE kitchen

**Best for: I/O -bound tasks
(network, disk, waiting)**

Multi -processing

Many separate kitchens, each with its own staff

**Best for: CPU-bound tasks
(heavy computation)**

Asynchronous (asyncio)

ONE chef juggling many pots at once

**Best for: Massive numbers
of I/O waits**

Multi -Threading: Shared Kitchen, Shared Resources

A **thread** is a lightweight worker living inside the same process , **sharing memory with every other thread** .

Threads are cheap to create and perfect for tasks that spend most of their time **WAITING** (I/O -bound work).

The Catch: The Global Interpreter Lock (GIL)

Only ONE thread can execute Python bytecode at a time. Threading does NOT speed up pure computation , but it DOES speed up I/O, because the GIL is released while a thread waits.

✓ Use threads when:

Downloading files, calling APIs, reading from disk, handling many slow client connections

Multi -Processing: Separate Kitchens, Separate Staff

A **Process** is a completely independant copy of your program , **it has it's own memory, own python interpreter and no shared GIL.**

This means TRUE parrallel execution across multiple CPU cores.

The Catch: Cost of Independence

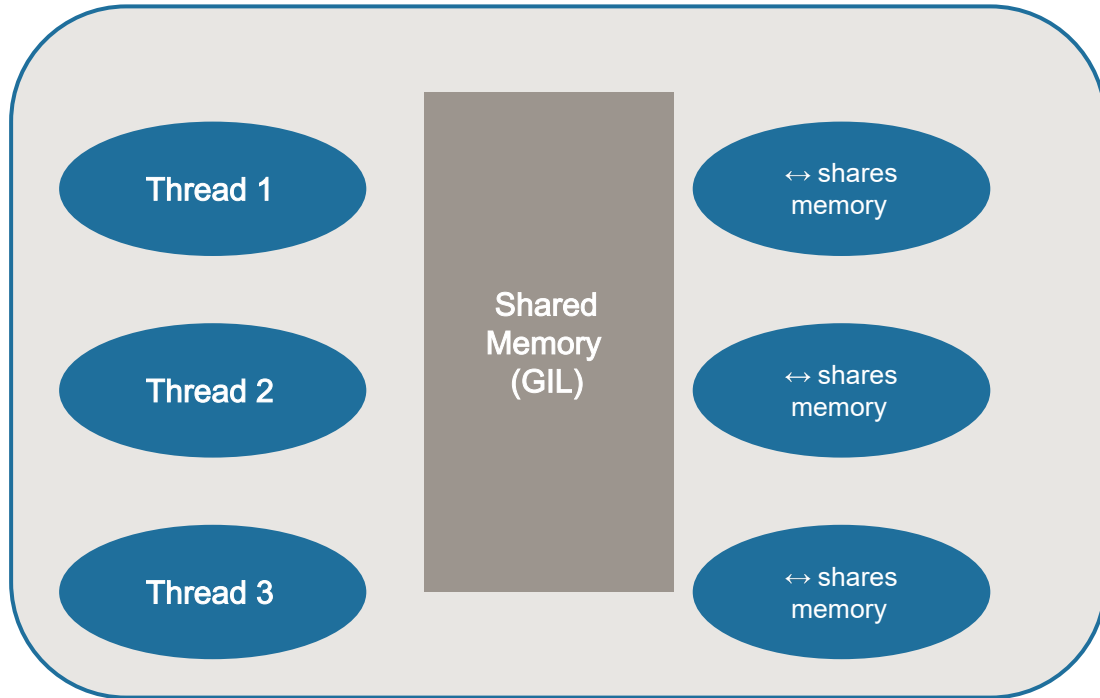
Each process has memory overhead and startup cost. Sharing data between processes is slow and requires special tools (Queues, pipes, shared memory, etc.)

✓ Use processes when:

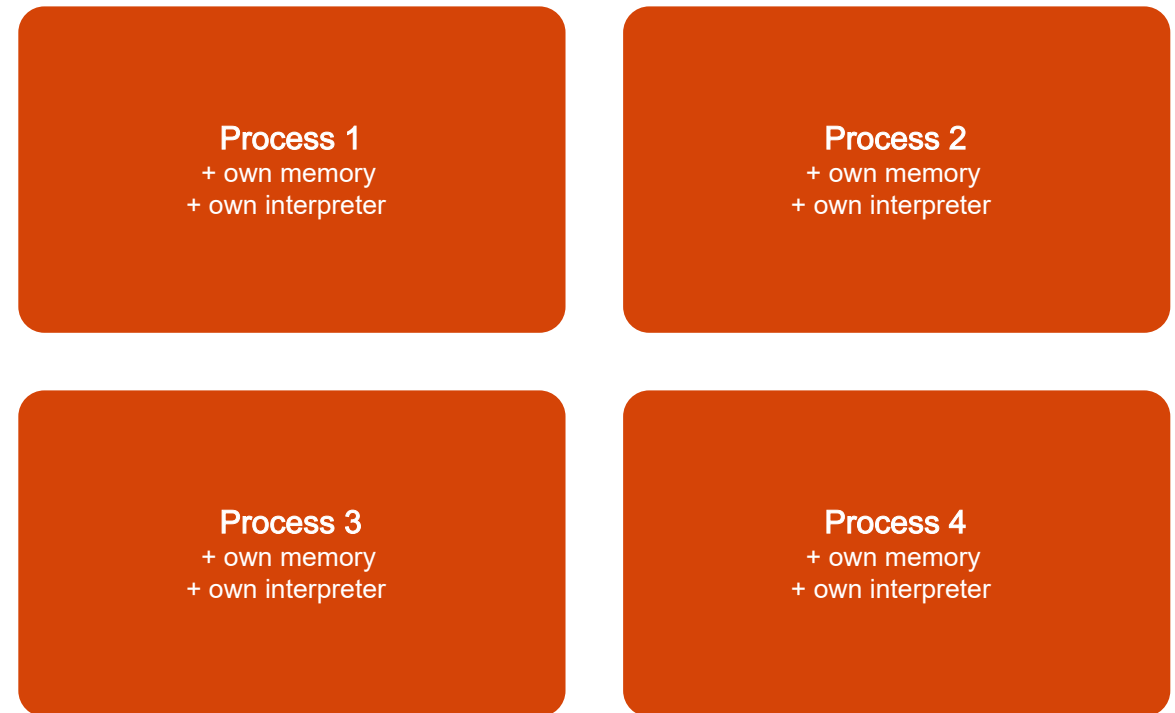
Image /video processing, simulations, machine learning training (anything CPU-hungry)

Threading vs. Multiprocessing

Multi -Threading



Multi -Processing



Threads are like roommates sharing one kitchen ; fast to coordinate, but they can only use **the stove (CPU) one at a time** .

Processes are like separate apartments ; each fully equipped, truly parallel, but **communication** means picking up the phone.

Multi-Threading: Code Snippet

```
import threading, time

def download(name):
    print(f'Starting {name}')
    time.sleep(2)           # simulated I/O wait – GIL is released here
    print(f'Finished {name}')

threads = [threading.Thread(target=download, args=(f'file{i}',)) for i in range(5)]
for t in threads: t.start()  # launch all 5 downloads at once
for t in threads: t.join()   # wait for all to finish → total time ≈ 2s, not 10s
```

Observation

5 simulated downloads × 2 seconds each.

Sequential: 10 seconds.

Threaded: ~2 seconds. The 5× speedup happens because `time.sleep()` releases the GIL, all five threads wait 'at the same time'.

Multi -Processing: Code Snippet

```
from multiprocessing import Process

def compute_square(n):
    result = n * n          # pure CPU work – no waiting involved
    print(f'{n} squared = {result}')

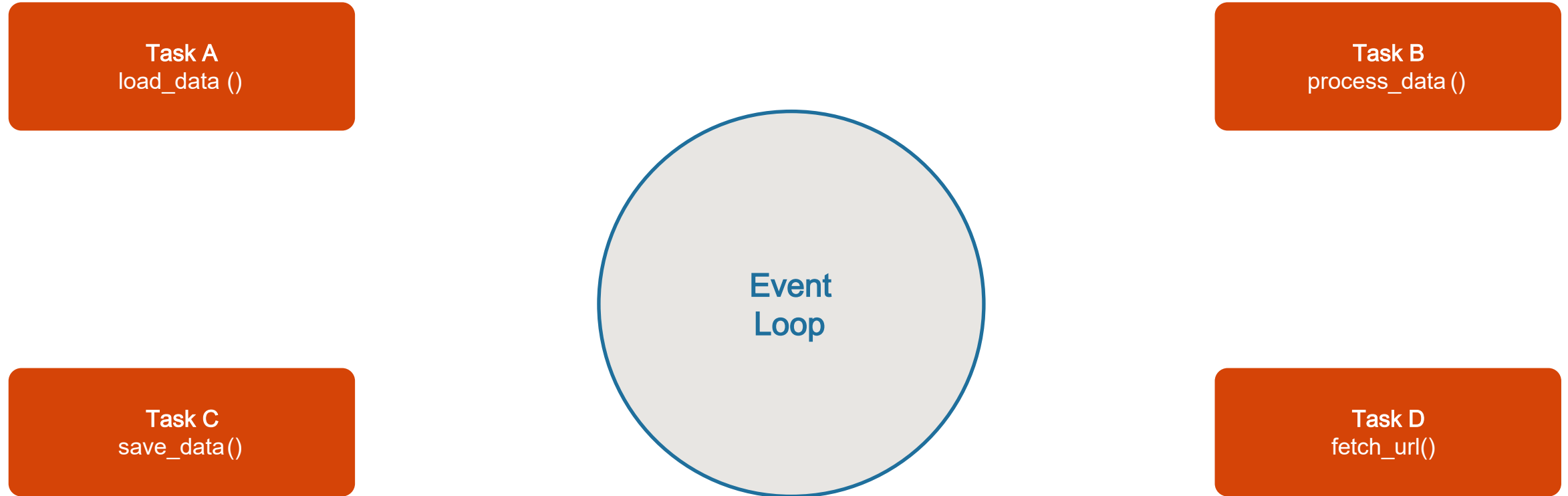
if __name__ == '__main__':
    numbers = range(1, 9)
    processes = [Process(target=compute_square, args=(n,)) for n in numbers]
    for p in processes: p.start()    # each runs in its own interpreter, own core
    for p in processes: p.join()
```

Observation

8 numbers, 8 processes, 4 cores. Total time $\approx 2\times$ the time for one process, since 8 tasks split across 4 cores means each core handles about 2. This is real parallel speedup (something threading can't give us here, because the GIL keeps CPU-bound Python work stuck on one core no matter how many threads you use)

Async Programming: The Juggler

A single thread runs an event loop that juggles many tasks. When a task hits 'await', it pauses and hands control back ; the loop runs another task in the meantime.



Key idea: when a coroutine hits 'await' (e.g. waiting for a network response), it yields control switches to another ready task. Nothing sits idle.

. The event loop immediately

Asyncio : Code Snippet

```
import asyncio

async def load_data():
    print('Loading...')
    await asyncio.sleep(2)          # non-blocking wait – loop is free to run other tasks
    print('Loaded')

async def process_data():
    print('Processing...')
    await asyncio.sleep(2)
    print('Processed')

async def main():
    await asyncio.gather(load_data(), process_data()) # run concurrently

asyncio.run(main())
```

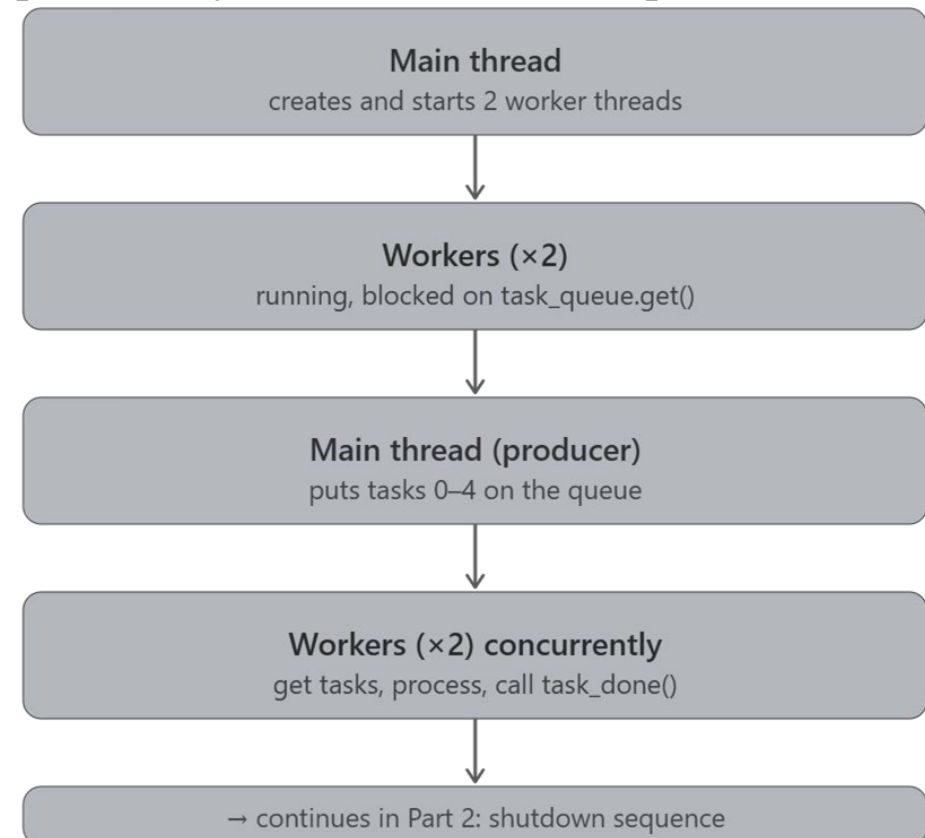
Observation

Two tasks, each 'taking' 2 seconds. Run sequentially: 4 seconds. With `asyncio.gather()`: ~2 seconds. All two sleep 'simultaneously' inside a single thread. No GIL contention, minimal overhead.

Task Queues: Decoupling Producers from Workers

This is a simple example of producer -consumer model. A queue lets one part of your program create work, while separate workers pick it up and execute it , independently, at their own pace.

Part 1: the concurrent dataflow. The main thread acts as the producer, **putting 5 tasks into a shared queue** . Two worker threads pull from that queue independently and each one sleeps 1 second per task to simulate work, then prints progress.

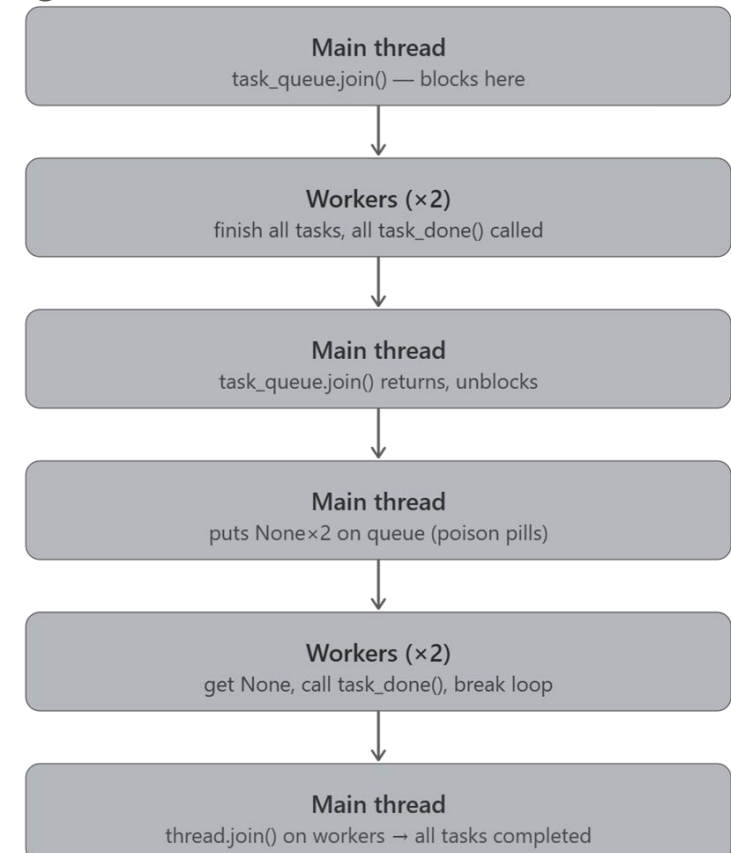


Task Queues: Decoupling Producers from Workers (Cont.)

Part 2: the shutdown sequence.

- `task_queue.join()` blocks the main thread until every task is marked done.
- Only then does the producer send two None sentinels / poison pill (one per worker) to signal "stop." Each worker sees its None and breaks its loop.
- Finally the main thread calls `thread.join()` on both workers and the program ends.

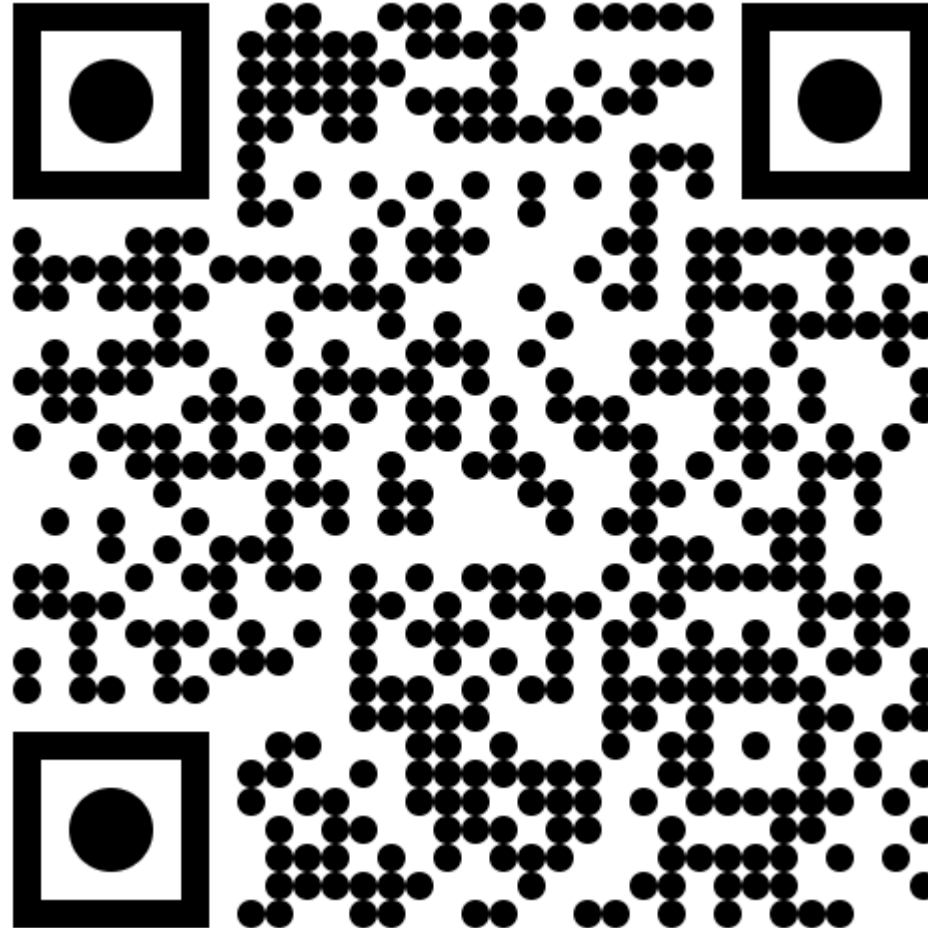
Producer-consumer queues power real systems: web servers queue requests, background -job systems queue emails and reports, and task schedulers queue delayed jobs. Workers scale independently, add more workers without touching the producer's code.



Pause / Break

See you in 15 minutes 😊!
Please be on time..

Received Feedback



Exercises

Solve Problems (1 -5) in “Week2_Exercise.ipynb “ and you have 45 minutes to finish all the tasks. *Don't forget to split your screen. Read the whole instructions first and answer the reflection questions in your own words.*

Problem 1: Multi-threading Simulation

Write a multi-threaded program that simulates downloading files.

Requirements:

- simulate downloading **five files**
- each download should take **two seconds**
- use the `threading` module
- print messages indicating when each download **starts** and **finishes**

Questions:

- What is the approximate total runtime?
- What would the runtime be if the downloads were executed sequentially?

Starter Code

```
python
import threading
import time
```

```
def download(file_name):
    # simulate download
    pass
```

```
files = ["file1", "file2", "file3", "file4", "file5"]
```

create threads here

Problem 2: Multi-processing Computation

Create a program that calculates the square of numbers from 1 to 8.

Requirements:

- Implement the program using multiprocessing
- Simulate a one-second delay for each computation
- Run computations in parallel

Questions:

- How many processes are created?

Since the Moodle is temporarily unavailable today, please push your in-class exercises on your private repository.

Key Takeaways

Concurrency $\neq$ more CPUs
It's about not wasting time while waiting

Threading $\rightarrow$ I/O -bound
GIL limits CPU work, but releases during waits

Multiprocessing $\rightarrow$ CPU-bound
True parallelism across cores, higher overhead

Asyncio $\rightarrow$ massive I/O scale
One thread, thousands of waiting tasks

Queues $\rightarrow$ decouple work
Producers and workers scale independently

Pick the tool that matches the bottleneck
Waiting? Threads/async. Computing? Processes.

Take-Home Assignment

Complete the following exercises at home. Remember to split your screen and solve the problems. You may always Google, check the syntax, resolve errors, and brainstorm, but **do not generate code or answers**. See the [submission instructions](#).

Problem 1: Comparing Threading and Multiprocessing

Create two versions of the same program:

- a threading version
- a multiprocessing version

The program **should**:

- compute factorial values
- simulate heavy computation using `time.sleep`

Numbers to process:

`10, 11, 12, 13`

Tasks:

- measure execution time for each version
- determine which approach is faster

Problem 2: Simple Task Scheduler

Create a basic task scheduler.

Requirements:

- Tasks should execute after a specified delay
- Store tasks in a queue
- Workers should periodically check the queue
- Execute tasks when the scheduled time arrives

Hints:

- The deadline is 28.06.2026 @ 23:59.
- See you next time when we dive into GPU Programming & Performance

Any Questions?

See you next time 😊

Dr. Ghadeer Mobasher

ghadeer.mobasher@srh-hochschulen.de

<https://www.linkedin.com/in/ghadeermobasher/>

LGS 6, 2. OG, Arc 213

